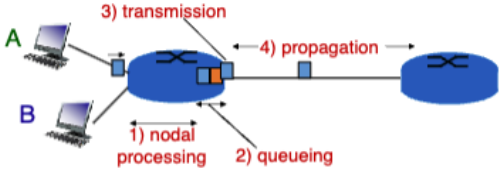
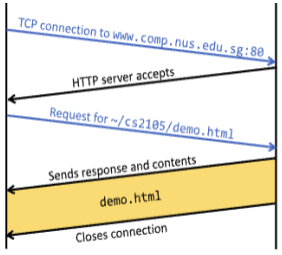
Www, email, ssh, rdp, ftp, ntp run over the Internet
Internet: a network of connected computing devices, infrastructure that connects hosts/end systems together (network edge: end hosts, servers, etc, network core: ISPs, routers, etc)
Hosts run network applications, applications communicate using protocols
Protocols define the 1. Format and order of messages exchanged among network entities and 2. Actions taken upon receiving or sending the messages
Hosts access the Internet through residential/institutional/mobile access networks
Wireless LANs (within building) aka WiFi | Wide-area wireless access aka 3G, 4G
How is data transmitted through the network
1. Circuit Switching: communication method where a dedicated communication path (or "circuit") is established between two devices for the entire duration of a communication session (call/data transfer).
2. **Packet Switching** (how the internet works now)
- host sending function: breaks app msg into smaller chinks (packets) of L bits and transmits pkts onto link at transmission rate R
- store&forward: packets are passed from one router to another across links on path from src to dest. entire packet must arrive at router before it can be transmitted on the next link (end to end delay = 2 * L/R)
- routing & addressing: routers determine route taken, each pkt needs to carry the src and dest info
- pkts share network resources, resources are used on demand, excess congestion is possible
How is the internet connected: each access ISP is connected to a global ISP, global ISPs are connected by IXPs (Internet Exchange Point) via peering links. Regional ISPs may arise as middlemen to connect access net to global ISPs.
IP addr & internet naming administered by Network Information Centre (NIC)
The Internet Society (ISOC) provides leadership in Internet related standards | Internet Architecture Board (IAB) has authority to issue and update technical standards regarding Internet protocols | Internet Engineering Task Force (IETF) - protocol engineering, development and standardisation arm of IAB → Internet standards published as RFCs (Request for Comments)



Processing delay: check bit errors, determine output link, typically < msec
Queueing delay: time waiting in queue for transmission, depends on congestion level of router
Transmission delay: packet length (L bits) / link bandwidth (R bps)
Propagation delay: length of physical link (d) / prop speed in medium (s) (~2×10^8m/s)
End-to-end delay: time taken for a pkt to travel from src to dest (all delays combined)
`traceroute` program displays path from src to dest & measures delay from src to each router along the path
Throughput: how many bits can be transmitted per unit time

Application Layer

Client-server: server wait for incoming req, client initiate contact first then req
P2P: no always on server, arbitrary end systems directly communicate. Peers req svc from other peers, provide svc in return to other peers. highly scalable, hard to manage
App-layer protocols define: type of msg xchanged, rules, msg syntax, semantics
Identifying network process: 1. IP Address (globally unique) identifies host → IPv4:32 bit number in dotted decimal notation | IPv6: 128-bit in hexadecimal notation 2. Port Number (locally unique) identifies process → 16 bit number (but 1 to 1023 reserved)
URL: Uniform Resource Locator (explains how to locate resources) | HTTP: Hypertext Transfer Protocol (explains how to access resources) uses TCP

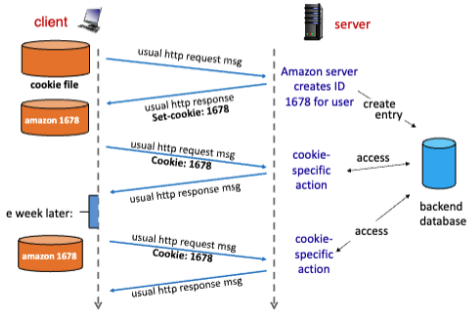


RTT: Round Trip Time → Time for packet to travel from client to server and back
eg 1 HTML and 5 obj
HTTP/1.0 (Non persistent): RTT (TCP) + RTT + a + 5 * (2RTT + b)
HTTP/1.1 (Persistent): RTT (TCP) + RTT + a + 5 * (RTT + b)
HTTP/1.1 (Persistent & Pipelining): 1 RTT (TCP) + RTT + a + RTT + 5b
Pipelining: new req can be made even before receiving response of old req
Multiplexing: Response can come back in any order, even partially
HTTP/2.0 (Multiplexing): RTT + RTT + a + RTT + 5b

```
GET /~cs2105/demo.html HTTP/1.1 \r\n
Host: www.comp.nus.edu.sg \r\n
User-Agent: Mozilla/5.0 \r\n
Connection: close \r\n
\r\n

HTTP/1.1 200 OK \r\n
Date: Wed, 01 Jul 2015 08:47:52 GMT \r\n
Server: Apache/2.4.6 (Unix) OpenSSL/1.0.1m \r\n
Accept-Ranges: bytes \r\n
Connection: Keep-Alive \r\n
Content-Length: 73 \r\n
Content-Type: text/html \r\n
Keep-Alive: timeout=5, max=100 \r\n
\r\n
<!DOCTYPE html>
<html lang='en'>
…
```

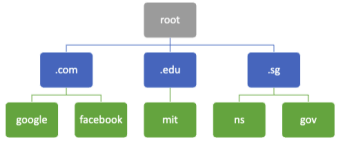
HTTP Status codes
`200 OK`: req success
`301 Moved Permanently`: new loc to follow
`304 Not Modified`: obj has not changed since specified date/time
`403 Forbidden`: Server declines to show webpage
`404 Not Found`: Requested Object not found
`500 Internet Server Error`: Unspecified error



Stateless vs Stateful: HTTP designed to be stateless, cookies maintain state
Cookie: attached to http messages in header, cookie file kept on user's host managed by user's browser

How to tell if resource has changed: dont send obj if client cache has up to date cached ver. Response contains no obj if cached copy is up to date (304 not modified)

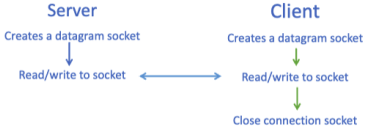
`telnet`: connect to a server over TCP, port 23 by default, can use it for any port (80 for HTTP) → manually type in requests to see servers raw response eg `telnet example.com 80` (then enter HTTP req) | `curl`: sends HTTP requests (and other protocol req) from the command line `curl https://example.com`
Domain Name System: translates betw host name and IP Address (client carry out DNS query → determine IP addr corresp. to server name prior to connection) runs over UDP/53 | Mapping betw host name & IP addr stored as Resource Records (RR)
`nslookup`: queries DNS servers to find out information `nslookup google.com`
`dig`: more detailed tool for doing DNS queries `dig comp.nus.edu.sg any`



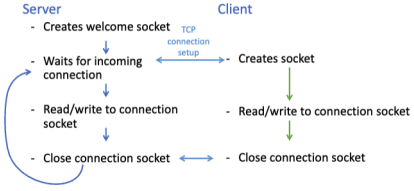
13 **root** servers
Top Level Domain servers: .com, .org, .uk, .sg
Authoritative servers: organisation's own DNS servers, provides authoritative hostname to IP mappings for org's named hosts
Local DNS server (does not belong to hierarchy): each ISP has one local DNS server, when query made it is sent here → retrieve name to addr. translation from local cache and acts as proxy, forwards query into hierarchy if answer is not found locally. Cached entries expire (TTL) & may be out-of-date. host change IP, may not propagate until all cache expires

Transport Layer

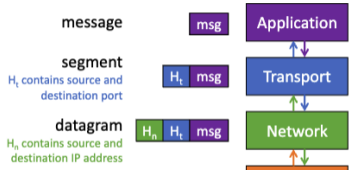
Standard port numbers: HTTP/80, POP/25, WoW/3724
Socket: abstraction interface between processes and transport layer protocols



UDP (User Datagram Protocol): datagram socket uses UDP → data sent as datagrams (packets). Only one socket is needed. Application creates each pkt, specifies dest IP and port and OS attaches src IP and port. Rcver identifies sender by extracting source IP and port



TCP (Transmission Control Protocol): data flow in continuous stream, process establish connection to another process, while connection is in place, data flows btw processes in continuous stream. TCP server: creates listening socket, when contacted by client, forks a new socket to comm. w/client | TCP client: creates socket to est. connection w/server (each connection has own socket instance)

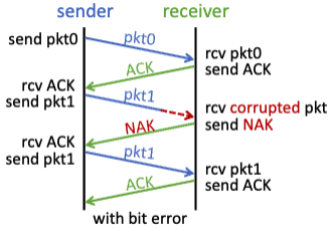


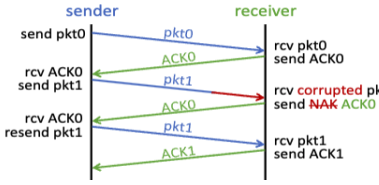
TCP has a "pipe" for communication: when one process want to send more bytes, it just writes data to the pipe (no need attach dest IP addr and port) but UDP need to form datagram packets explicitly and attach dest IP and port to every pkt

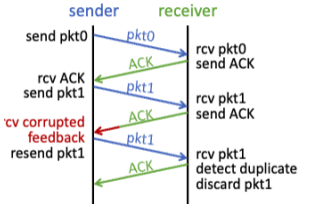
Sender side: breaks app msg into segments as needed, passes them to network layer
Receiver side: reassembles segments into message, passes to app layer
Packet switches (routers) in betw: check dest IP to decide routing

Reliable delivery transfer (rdt) models

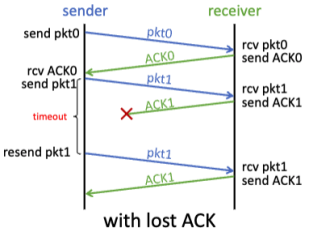
rdt 1.0: underlying channel is reliable
rdt 2.0: assume underlying channel may flip bits in pkts → use checksum to detect bit errors and implement ACK and NAK (stop & wait protocol)
rdt 2.1: add sequence number (fix if ACK/NAK corrupted problem)
rdt 2.2: replace NAK with ACK for last pkt received correctly (receiver must explicitly include seq # of pkt being ACKed) (duplicate ACKs at sender → retransmit current pkt)

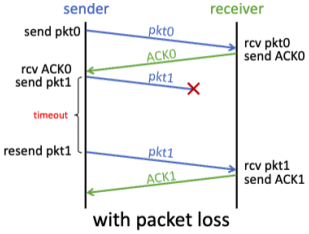






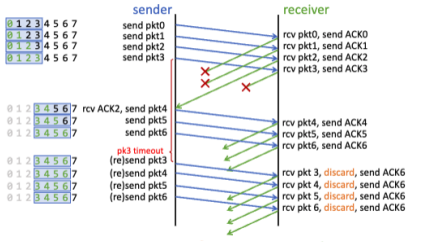
rdt 3.0: underlying channel may flip bits/lose pkts/incur arbitrarily long pkt delay
sender waits reasonable amt of time, retransmits if no ACK received till timeout



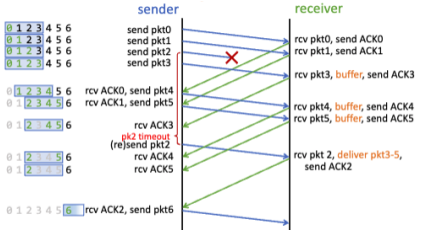


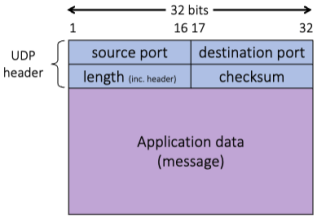
Utilisation: fraction of time link is actually being used
Stop & wait U = dtrans / (dtrans + RTT) | throughput = L/(RTT + dtrans)

Go-Back-N: Sender → can have up to N unACKed pkts in pipeline, insert k-bits seq no. in pkt header, use sliding window to keep track of unACKed pkts, keep timer for oldest unACKed pkt, timeout(n): retransmit pkt n and all subsequent pkts in window | Receiver → only ACK pkts that arrive in order, discard out-of-order pkts and ACK the last in-order seq #

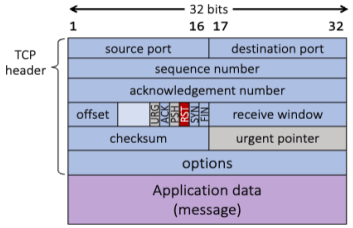


Selective Repeat: Sender → maintains timer for each unACKed pkt, when timer expire, retransmit only that pkt | Receiver → individually ack all correctly rcved pkts, buffers out-of-order pkts for eventual in-order delivery to upper layer





UDP segment structure (left)
Checksum → receiver verifies checksum by adding all up, if all 1s → pass



seq no: counts bytes not seg #, is the byte number of first byte of data in the segment | **ACK**: indicates if ACK # is valid, is the seq # of next byte of data expected (TCP only ACKs up to missing byte → cumulative ACK) → TCP has bi-directional data flow, ACKs are pigybacked on data segment | TCP Acking: 1. In order segment: a. If all data alr acked, wait 500ms to send ack again b. Outstanding segment not acked → send ack 2. Out of order segment: send duplicate ack of expected byte 3. Segment partial/completely fills gap: send cum. ack | 3 dupe acks → resend segment

MSS (Maximum Segment Size) – does not include header (MTU includes header)

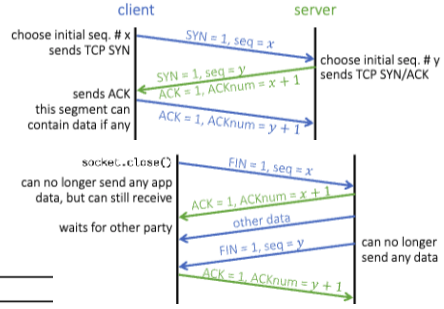
TCP est. connection (**SYN**)

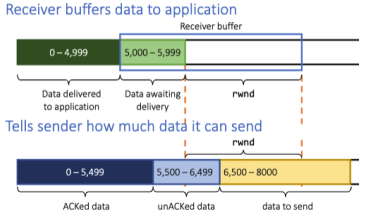
TCP closing connection (**FIN**)

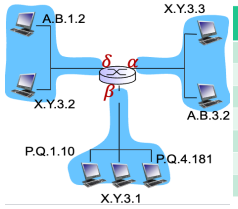
→ send segments w FIN bit set

→ no more sending data after FIN but can still rcv

TCP flow control (**receive window**)





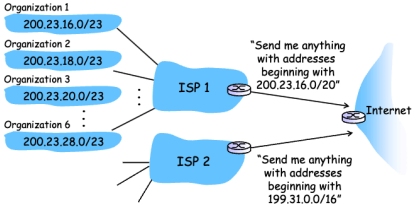


Destination	Output Link
A.B.1.2	δ
A.B.3.2	α
P.Q.1.10	β
P.Q.4.181	β
X.Y.3.1	β
X.Y.3.2	δ
X.Y.3.3	α

Network Layer

Subnet: a network formed by a group of "directly" interconnected hosts → hosts in same subnet can physically reach each other without intervening router, they connect to outside world through a router. they have same network prefix of IP addr. a.b.c.d/x where x is the no. of bits in subnet prefix of IP addr. Subnet mask: used to determin which subnet an IP belongs to (should be all 1s then all 0s)

Hierarchical Addressing →



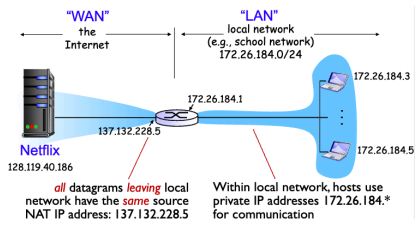
Special IP Addresses:

0.0.0.0/8: non routable meta-addr

127.0.0.0/8: loopback addr. A datagram sent here loops back inside host

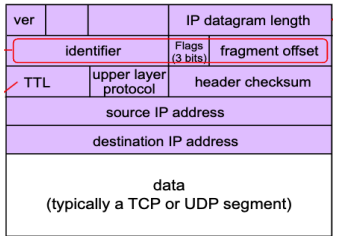
10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16: **private** addresses, not globally unique, used in organisations, unis and homes, not routable on internet but are routable within an organisation

255.255.255.255/32: broadcast addr.



NAT translation table	
WAN side	LAN side
137.132.228.5, 5001	172.26.184.3, 3213
137.132.228.5, 5002	172.26.184.5, 3213

172.26.184.3 sends datagram to 128.119.40.186, 80 → NAT router changes datagram src addr, port, updates table → reply arrives, dest add is 137.132.228.5, 5001, NAT router changes datagram dest addr to 172.26.184.3, 3213



IP datagram length: header + data

TTL: no of remaining hops

Flag: 0 if this is last segment

Identifier: same segment same id

Fragment offset: eg 3 fragments: 480B, 480B, 220B, offset: 0, 480/8 = 60, 960/8 = 120 respectively

DHCP (Dynamic Host Config Protocol): allows host to dynamically obtain IP from DHCP server when it joins network (runs over UDP, server port 67, client port 68) (app layer protocol)

Host broadcast DHCP discover msg → DHCP server responds w DHCP offer msg → Host req IP addr DHCP req msg → DHCP server sends addr DHCP ACK msg

Longest Prefix Match: when looking for forwarding table entry for given dest addr. use longest addr. prefix that matches dest addr.

Intra-AS routing: Finds good path betw 2 routes within an AS eg RIP (Routing Information Protocol): implements Distance Vector algo, using hop count as metric and exchange routing table every 30s over UDP/520 (app layer protocol)

DV algo: distributed → each node rcv info from one/+ of its directly attached neighbours, performs calc, distributes result back ; iterative: process cont until no more info avail to be exchanged betw neighbours ; async: doesnt req all nodes operate in lock step w e/o

ICMP (Internet Control Message Protocol): used by hosts & routers to communicate network-level info → error reporting (unreachable host/network/port/protocol), echo request (`ping`: sees if remote host will respond). ICMP msgs carried in IP datagrams, ICMP header starts aft IP header | ICMP header: type + code + checksum + others

Type	Code	Description
8	0	echo request (ping)
0	0	echo reply (ping)
3	1	dest host unreachable
3	3	dest port unreachable
11	0	TTL expired
12	0	bad IP header

`traceroute`: sends a series of small pkts across a network with different TTL, attempts to display route msg take to get to remote host

Link Layer

Broadcast link (shared medium) vs Point-to-point link (sender & rcver connected)

Random Access Protocols: transmit at full channel data rate R

Slotted ALOHA: all frames are of L bits, time divided into slots of eql length (L/R), nodes start to transmit only at beginning of a slot | when node has fresh frame to send, wait until beginning of next slot and transmit entire frame in the slot (not collision free, when many nodes active max efficiency 37%)

Pure ALOHA: transmit entire frame immediately (no slot), if collision, wait 1 frame transmission time and retransmit frame with probability p until success (not collision free, max efficiency 18%)

CSMA (Carrier Sense Multiple Access): listen before transmit, if channel idle, transmit entire frame (collisions can still occur due to prop delay)

CSMA/CD (Collision Detection): same as ^ + if collision detected, abort transmission, retransmit after a random delay (wait for 1 frame transmission time, retransmit frame with probability p until success → binary exponential backoff: after mth collision, choose K at random from {0, 1, …, 2^m -1} and wait K time units before retransmission.

p = $1/2^m$) *for ethernet 1 time unit = 512 bit transmission times* → minimum frame size = 2*prop delay*link rate

Taking Turns Protocols

Polling (one node needs to be master node): Master node polls each of the nodes in a round-robin fashion (not decentralised, master node is a single pt of failure)

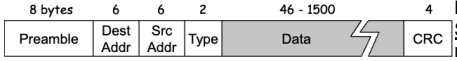
Token Passing: token is passed from one node to the next. When node rcv token, hold on to it if it needs to transmit frames, sends up to max no of frames then forwards token to next node, else forward to next node

Channel Partitioning Protocols

TDMA (time division multiple access): Each node gets fixed length time slots in each round of access (length of time slot = data frame transmission time) (max throughput = R/N, inefficient)

FDMA (frequency division multiple access): channel spectrum divided into freq bands, each node assigned a fixed frequency band, unused transmission time in freq bands go idle (max throughput = R/N, inefficient)

Error Detection: Parity Checking → can detect and correct single bit errors, can detect any two-bit error in data | Cyclic Redundancy Check (CRC) → add r number (no of digit in G) of 0s to D, XOR (0 XOR 1 = 1) with G, find remainder to get R

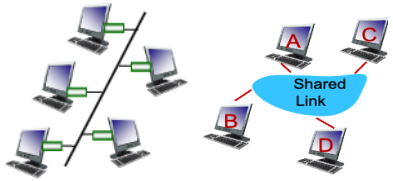


Ethernet Frame Structure

Src & dst MAC address: if NIC rcv frame w/matching dst addr or with broadcast addr, it passes data in frame to network layer protocol, else it discards it | data: max 1500 (link MTU) min 46 (ensure collision alw detected)

CRC: corrupt frame will be dropped | Type: indicates higher layer protocol | Preamble: 7 bytes with pattern 10101010 followed by 1 byte of 10101011 (sync clock rates)

Ethernet's multiple access protocol: CSMA/CD with exponential backoff



Bus v Star topology (Star prevalent)

Ethernet Switch: Link layer device used in LAN → examines incoming frame's MAC addr, selectively forward frame to one-or-more outgoing links

Switch table

MAC addr	Interface	TTL
A	1	60
D	3	60

Switch learns which hosts can be reached through which interfaces, records sender/location pair in switch table. When frame received at switch → record incoming link, MAC addr of sender → index switch table using MAC dest addr → if entry found for dest → if dest on segment from which frame arr, drop frame, else forward frame indicated by entry → else broadcast (forward on all interfaces except arriving interface)

When adapter receive frame, checks if dest MAX addr of frame matches its own MAC addr → yes: extracts enclosed datagram and passes to protocol stack → no: discard

MAC addr 48 bits (permanent unlike IP), broadcast addr: FF-FF-FF-FF-FF-FF

ARP (Address Resolution Protocol): translating from IP to MAC. each IP node has an ARP table storing mapping of IP and MAC in same subnet <IP add; MAC addr; TTL>

Sending frame in Same Subnet

If A knows B's MAC addr from ARP table: create frame with B's MAC addr and send it, only B will process this frame, other nodes will ignore it

If A doesnt know B's MAC addr: A broadcasts ARP query pkt containing B's IP addr, dest MAC addr set to FF-FF-FF-FF-FF-FF → all other nodes in same subnet will rcv this ARP query pkt but only B will reply → B replies with MAC addr in reply frame → A cache B's IP-to-MAX addr mapping in its ARP table

Sending frame to Another Subnet

A creates IP datagram with IP src A, dest B → A creates link-layer frame with R's MAC address as dest addr → frame sent from A to R → R removes datagram, passes up to IP → R forwards datagram with IP src A dest B → R creates link-layer frame with B's MAC addr as dest addr

Network Security

Symmetric Key Cryptography: sender and receiver use same key ($K_A = K_B$)

The key C_s is unique to a pair of individuals (hard to agree on key if never "met")

Caesar's cipher: fixed shift of alphabet (eg right shift by 3)

Monoalphabetic cipher: substitute one letter for another (26! mappings) can be broken by statistical analysis

Polyalphabetic encryption: use multiple mappings instead of only 1, define a cycling pattern, for each new plaintext symbol, use subsequent substitution pattern

Block Ciphers: message to be encrypted is processed in blocks of K bits, cipher uses 1-to1 mapping. No of keys: 2^K!

Asymmetric Key Cryptography (Public key cryptography): different key

Sender uses receiver's public encryption key, receiver uses private decryption key

RSA algorithm: message (bit pattern) can be represented by integer

Choose 2 large prime p, q → compute n = pq, z = (p-1)(q-1) → choose e (e < n) that has no common factors with z → choose d such that ed - 1 is exactly divisible by z

Public key is (n, e) and private key is (n, d)

To encrypt m: $c = m^e \bmod n$ To decrypt: $c^d \bmod n$

→ can use RSA to send encrypted symmetric session key, then use symmetric key

Cryptographic Hash Function: hash func computationally infeasible to find any 2 diff msgs x and y such that H(x) = H(y)

Message authentication code: sender & rcver share Authentication key s, to ensure message integrity, send (m, H(m + s))

Password hashing: pw hashed and stored (cannot be recovered, only reset)

Digital Signatures: must be verifiable and unforgeable → can sign message m with private key and create signature $K_B^-(m)$

Public key distribution methods: PSA, publicly available directory & public key infrastructure (PKI) → certificate (identify of owner, public key, validity time window, signature) + certificate authority (CA) (issues and signs digital certs) When rcver want sender pub. key: get sender's cert, apply CA's pub. key to it to get sender's public key

Firewall: isolates organisation's internal net from larger internet, allowing some pkts to pass, blocking others | **stateless pkt filters**: internal network connected to internet via router firewall, router filters based on src and dst IP, TCP/UDP src and dst port no, ICMP msg type, TCP SYN and ACK bits (ACL Access control lists: table to rules applied to incoming pkts → action condition pairs) | **stateful pkt filters** | **app gateways**